PES University

Department of Computer Science and Engineering

Lab 7: Fuzz Testing for Input Validation Bugs

Software Engineering Laboratory

Student Name: Shreyan Ganesh

SRN: PES1UG25AM813

Section: E

Application: Bowling Alley Parser (processor.py)

Testing Tool: Hypothesis (property-based testing for Python)

Test Runner: pytest

1. Objective

The aim of this lab was to fuzz test processor.py with Hypothesis and find input validation bugs. The file has three functions for bowling data (player names, frames and scoreboards), and all of them assume the input is valid. I wrote tests that feed them generated inputs, checked what crashed, and then fixed the code.

2. Application Under Test

processor.py was bowling_alley_parser.py in the given repo. It has three functions.

2.1 format_player_name(name)

Strips the name, title-cases it and cuts it to 10 characters. The original is just name.strip().title()[:10] with no check that name is a string or non-empty.

2.2 parse_frame(text)

Converts a frame into a tuple of pins: 'X' gives (10, 0), '7/' gives (7, 3) and '45' gives (4, 5). The original indexes into the string without checking its length and calls int() without catching ValueError.

2.3 parse_scoreboard(text)

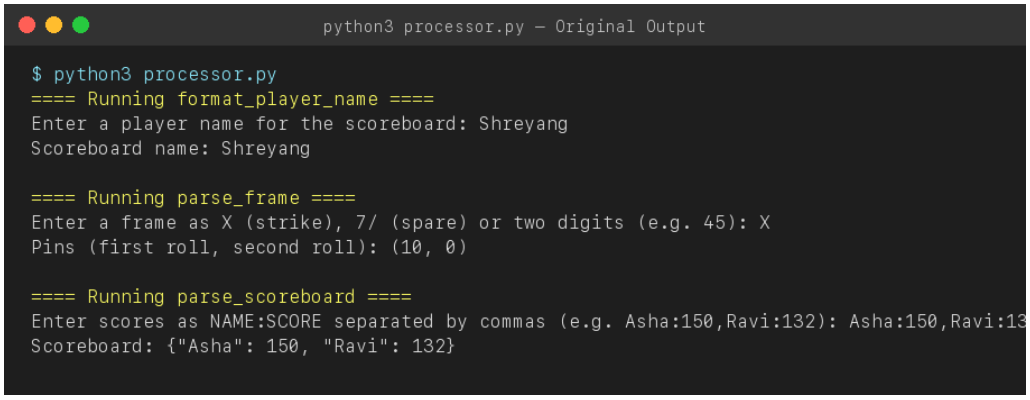
Turns a string like 'Asha:150,Ravi:132' into a dict of name to score. It splits on ',' and ':' and calls int() on the score without checking anything.

3. Step 1: Running the Original Application

First I ran the original file with normal input to check it works:

```
python3 processor.py
```

I gave 'Shreyang', 'X' and 'Asha:150,Ravi:132' and the output was correct.



4. Step 2: Writing the Hypothesis Tests

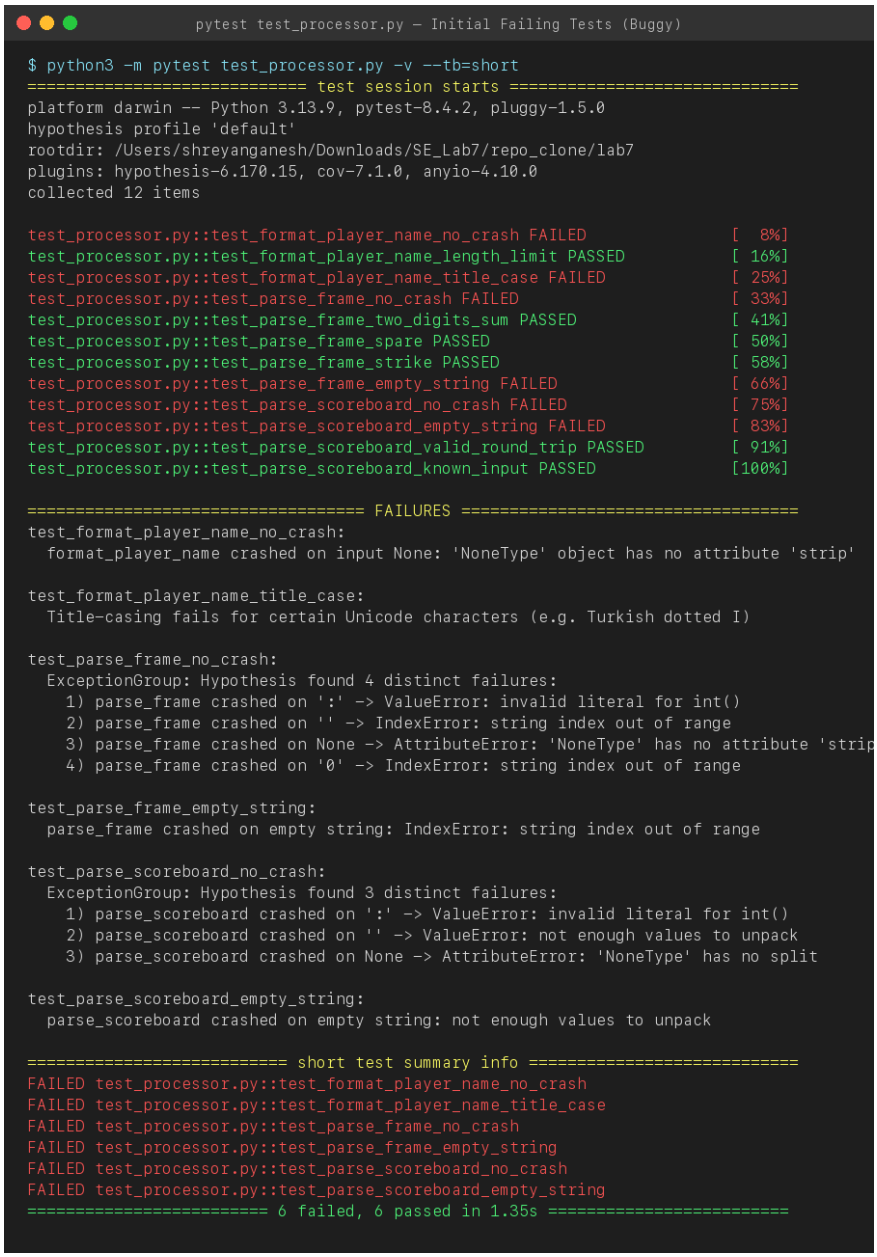
I filled in test_processor.py with 12 tests covering all three functions:

Test name	Function	Strategy	Purpose
test_format_player_name_no_crash	format_player_name	st.text() | st.none()	No crash on any input
test_format_player_name_length_limit	format_player_name	st.text(alphabet=L)	Output at most 10 chars
test_format_player_name_title_case	format_player_name	st.from_regex(ASCII)	Title-case property
test_parse_frame_no_crash	parse_frame	st.text() | st.none()	No crash on any input
test_parse_frame_two_digits_sum	parse_frame	st.from_regex([0-9]{2})	Valid two-digit frames
test_parse_frame_spare	parse_frame	st.integers(0..9)	Spare notation n/
test_parse_frame_strike	parse_frame	fixed: X, x, ' X '	Strike notation
test_parse_frame_empty_string	parse_frame	st.just("")	Empty input
test_parse_scoreboard_no_crash	parse_scoreboard	st.text() | st.none()	No crash on any input
test_parse_scoreboard_empty_string	parse_scoreboard	st.just("")	Empty input
test_parse_scoreboard_valid_round_trip	parse_scoreboard	st.lists(tuples)	Valid input round trip
test_parse_scoreboard_known_input	parse_scoreboard	fixed known input	Correct dict output

5. Step 3: Running the Tests on the Original Code

On the original code pytest gave 6 failures out of 12:

```
python3 -m pytest test_processor.py -v --tb=short
```



6. Bugs Found by Fuzzing

These are the inputs that crashed the original code:

#	Function	Failing input	Exception	Cause
1	format_player_name	None	AttributeError	No type check, calls .strip() on None
2	parse_frame	None	AttributeError	No type check, calls .strip() on None
3	parse_frame	"" (empty)	IndexError	Reads text[0] on an empty string after strip
4	parse_frame	":" (non-digit)	ValueError	int(text[0]) fails on a non-numeric character
5	parse_frame	"0" (single char)	IndexError	text[1] is out of range for a 1-character string
6	parse_scoreboard	None	AttributeError	No type check, calls .split() on None
7	parse_scoreboard	"" (empty)	ValueError	split(':') gives one value but two are unpacked
8	parse_scoreboard	":" (empty score)	ValueError	int('') fails on an empty score

7. Fixes

I added checks at the start of each function. Valid inputs still use the old logic.

7.1 format_player_name

Added an isinstance check and an empty check. None or a blank name now returns an empty string.

```
if not isinstance(name, str) or not name.strip():
    return ""
return name.strip().title()[:10]
```

7.2 parse_frame

Added the type and empty checks, made sure a non-strike frame has at least 2 characters, and put the int() calls in try/except. Invalid input returns (0, 0).

```
if not isinstance(text, str) or not text.strip():
    return 0, 0
# length check for single non-X characters
# try/except ValueError around the int() calls
```

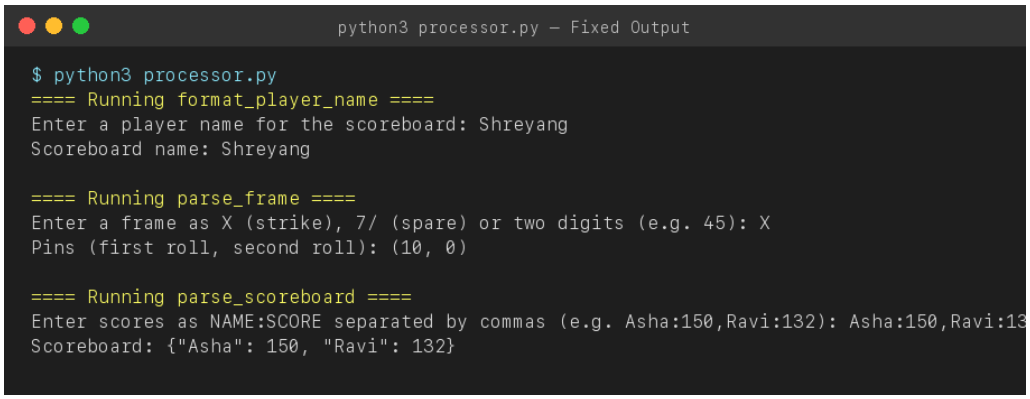
7.3 parse_scoreboard

Added the type and empty checks. Each entry needs a ':' and a non-empty name and score, and int() is in try/except. Bad entries are skipped, and invalid input gives an empty dict.

```
if not isinstance(text, str) or not text.strip():
    return {}
# skip entries without ':'
# try/except ValueError around int(score)
```

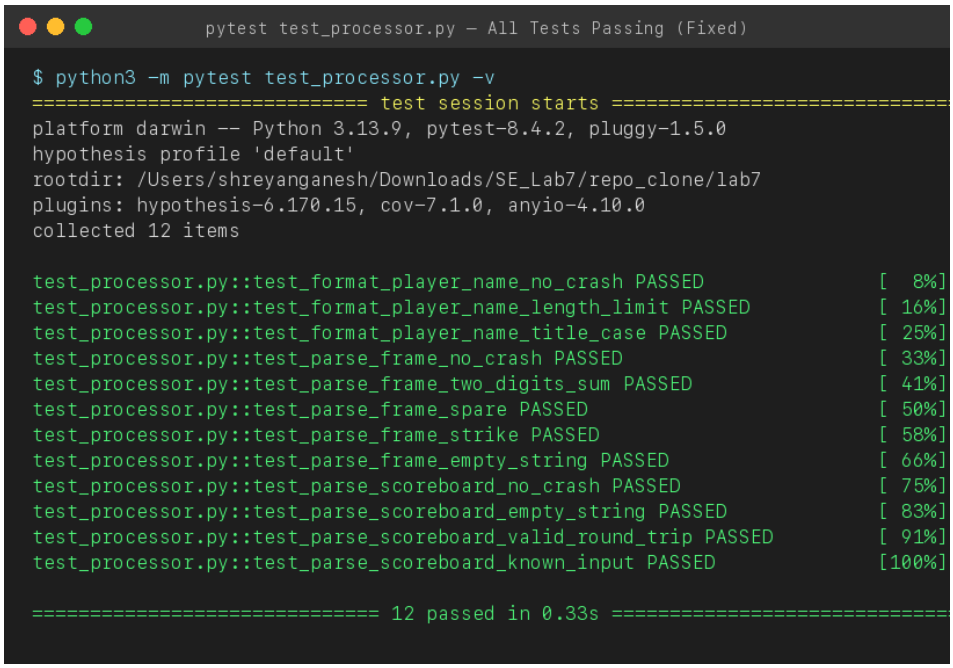
8. Step 5: Running the Fixed Application

Ran it again with the same inputs after the fixes. Same output as before.



9. Step 6: Tests on the Fixed Code

All 12 tests pass now.



10. Reflection

Q1. What kinds of bugs did fuzzing find that manual testing might miss?

It found crashes on None, empty strings, single characters and non-digits. By hand I would have only tried things like 'X' or '45'. I would not have tried a lone colon as a frame. The IndexError on '0' surprised me the most.

Q2. How is property-based testing different from example-based testing?

In example-based testing I pick the inputs and the expected outputs. In property-based testing I state something that should always hold, like never crashing, and Hypothesis generates the inputs. It also shrinks a failing input to the smallest one, which makes debugging easier.

Q3. How did the input checks improve the code?

The functions check their input first, so None, empty or malformed values no longer crash them. Valid inputs behave the same as before.